AI Investment Committee

AAI-520 Final Team Project, Group 7: Michael Valderrama, Guna Pasupathy, Mina Habib

Repository: https://github.com/ibucketbranch/AAI-520-Final-Project

Our project is a small "investment committee" made of AI agents. You give it a stock symbol. A planner decides what to research, three tools collect earnings, news, and market data, a Bull analyst and a Bear analyst argue from that evidence, an evaluator checks every claim against the evidence it cites, and the committee writes a short memo with a verdict. A memory note from each run is read by the planner on the next run.

This is a class project to show agent design and workflow patterns. **It is not financial advice.**

This is the **light** version. It calls the functions in our package and explains the results. The code itself lives in `src/investment_committee/` in the repository. A full version of this notebook prints that code inline after every section.

How to run it

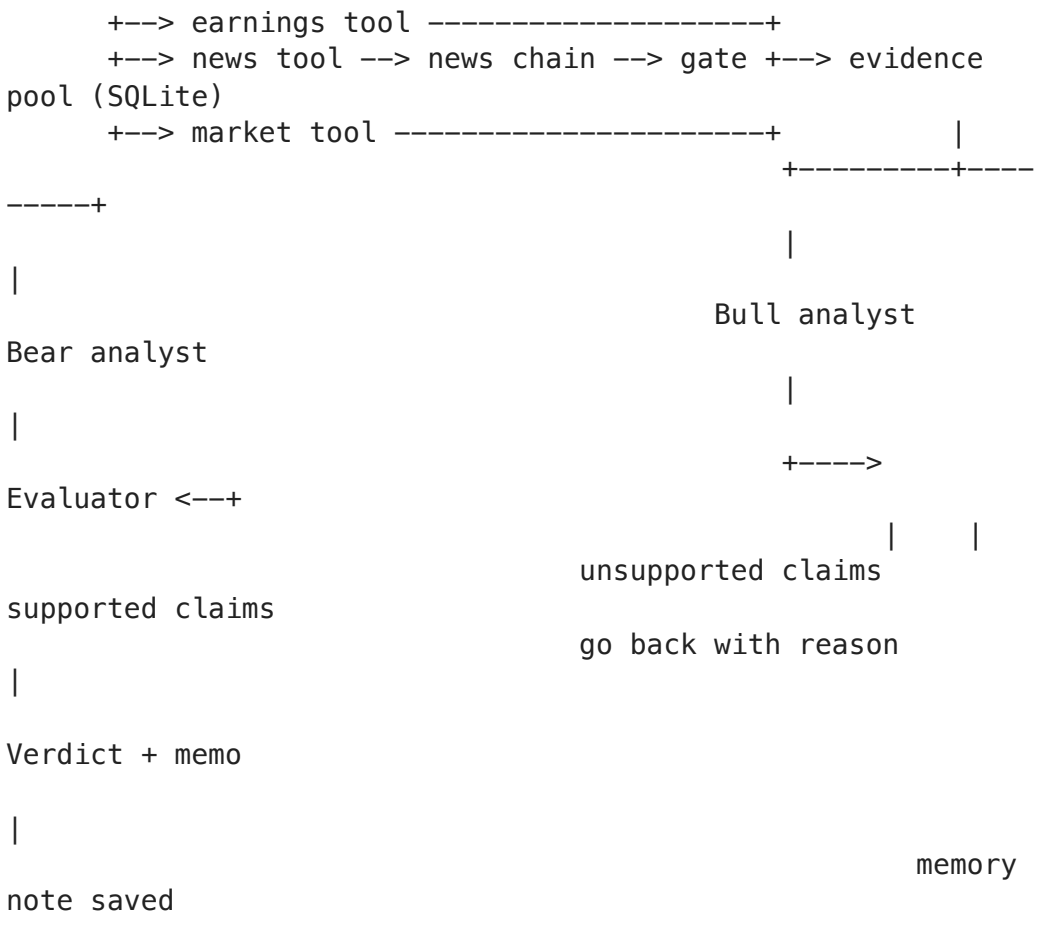
1. From the repo root: `pip install -r requirements.txt` (or `make setup`). The setup cell adds `src/` to Python's path, so the package works even if you skip `pip install -e .`.
2. Optional: copy `.env.example` to `.env` and add `OPENAI_API_KEY` (and `NEWSAPI_KEY` for news). Without keys everything still runs: the planner, analysts, and evaluator fall back to their rule-based versions.
3. `LIVE = True` (below) uses real market and news data, so it needs internet. `LIVE = False` uses the small sample files in `tests/fixtures/`.

The outputs saved in this notebook come from one live run with an OpenAI key. Live data and model answers change from run to run, so a re-run will show different numbers.

1. Agent design and workflows

The committee gives a short opinion on a stock, backed by evidence, and says what it could not check.

```
stock symbol
     |
  Planner  <-------------- memory note from the last run
     |
```



ChatGPT was used for formatting this text diagram

The agents

Agent	What it does
Planner	Picks which sources to research and in what order. Reads the last run's memory note.
Tools (earnings, news, market)	Collect real data with yfinance and NewsAPI and turn it into `Evidence` records.
Bull and Bear analysts	Each writes 1 to 5 claims for its side. Every claim must cite evidence ids from the pool.
Evaluator	Checks each claim against the evidence it cites and gives a reason when a claim is not supported.
Chair (verdict)	Reads only the supported claims and picks bull, bear, or none.

The three workflow patterns (from Anthropic, 2024)

1. **Prompt chaining:** the news chain runs ingest, preprocess, classify, gate, extract, and summarize, one step after another (section 3).
2. **Routing:** each planned step is sent to its own specialist tool (section 2).

3. **Evaluator-optimizer:** the analysts write claims, the evaluator judges them, and rejected claims go back to be rewritten (section 5).

Design choices we made on purpose

- **Everything cites evidence.** A claim without a valid evidence id is rejected, so the memo can always be traced back to data.
- **Every agent has a fallback.** No API key, a failed call, or a bad model answer drops back to a rule-based version and the run records why. A broken data source does not stop the run.
- **One database as the handoff.** Each stage reads the previous stage's results from SQLite by run id. This is also what makes the with/without-review experiment fair (section 7).

Setup

This cell finds the repo folder, makes the package in `src/` importable, checks that the spaCy English model is installed (the news chain needs it for named entities), and loads the API keys from `.env` if there is one. It also starts a fresh database file just for this notebook.

In [1]:
```
import subprocess
import sys
from contextlib import closing
from pathlib import Path

import pandas as pd
import spacy
from IPython.display import display

# Find the repo root (the folder with pyproject.toml), so paths work from
any folder.
ROOT = Path.cwd()
while not (ROOT / "pyproject.toml").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

# Use the package straight from src/, so this works even without "pip
install -e .".
sys.path.insert(0, str(ROOT / "src"))

# The news chain uses spaCy's small English model. Download it once if it
is missing.
# If the download fails (no internet), use a blank pipeline: the chain
still runs,
# it just finds no named entities.
try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    try:
        subprocess.run([sys.executable, "-m", "spacy", "download",
```

```
"en_core_web_sm"], check=True)
        nlp = spacy.load("en_core_web_sm")
    except (subprocess.CalledProcessError, OSError):
        print("Could not get the spaCy model, so entity extraction will
find nothing.")
        nlp = spacy.blank("en")

from investment_committee.config import load_settings

settings = load_settings(str(ROOT / ".env"))
HAVE_OPENAI = bool(settings.values.get("OPENAI_API_KEY"))
HAVE_NEWSAPI = bool(settings.values.get("NEWSAPI_KEY"))

LIVE = True

# "openai" uses the models; "template" is the rule-based fallback used
without a key.
PROVIDER = "openai" if HAVE_OPENAI else "template"

# A fresh database just for this notebook, so old runs do not mix in.
DB_PATH = ROOT / "data" / "final_notebook.db"
DB_PATH.parent.mkdir(exist_ok=True)
if DB_PATH.exists():
    DB_PATH.unlink()

print("Repo folder:     ", ROOT.name)
print("Live data:       ", LIVE)
print("OpenAI key found:", HAVE_OPENAI, "->", PROVIDER)
print("NewsAPI key found:", HAVE_NEWSAPI)
```

```
Repo folder:      aai520-final
Live data:        True
OpenAI key found: True -> openai
NewsAPI key found: True
```

In [2]:

```
import inspect

from investment_committee.workflows.chaining import clean_text

pd.set_option("display.max_colwidth", 200)


def plain(value):
    """Make text plain ASCII for display (news and model text can have
curly quotes)."""
    if isinstance(value, str):
        return "\n".join(clean_text(line) for line in value.splitlines())
    return value


def table(rows):
    """Show a list of dicts as a table, with plain text."""
    frame = pd.DataFrame(rows)
    if not frame.empty:
        frame = frame.map(plain)
    return frame
```

```
def show_code(*functions):
    """Print the real source code of package functions, with the file
name."""
    for function in functions:
        path = inspect.getsourcefile(function)
        short = path.split("src/")[-1] if "src/" in path else path
        lines, start = inspect.getsourcelines(function)
        print(f"# ---- {short}, line {start}")
        print("".join(lines))
```

In [3]:

```
import json

from investment_committee.models import Evidence
from investment_committee.tools.earnings import gather_earnings
from investment_committee.tools.market import gather_market
from investment_committee.tools.news import gather_news
from investment_committee.workflows.chaining import RawArticle,
processed_to_evidence, run_chain

SAMPLE_EVIDENCE = json.loads((ROOT /
"tests/fixtures/sample_evidence.json").read_text())
SAMPLE_ARTICLES = json.loads((ROOT /
"tests/fixtures/sample_articles.json").read_text())


def sample_gatherer(source):
    """Offline stand-in for a tool: the sample evidence rows for one
source."""

    def gather(run):
        rows = []
        for item in SAMPLE_EVIDENCE["evidence"]:
            if item["source"] == source:
                rows.append(
                    Evidence(
                        run_id=run.run_id,
                        symbol=run.symbol,
                        source=source,
                        title=item["title"],
                        content=item["content"],
                        as_of=item["as_of"],
                        reference="sample data",
                    )
                )
        return rows

    return gather


def sample_news(run):
    """Offline stand-in for the news tool: the saved sample articles for
this stock."""
    articles = []
    for item in SAMPLE_ARTICLES:
```

```
        if item["symbol"] == run.symbol:
            articles.append(RawArticle(**item))
    return articles


def chained_news(run):
    """News tool plus the news chain, so news comes back as Evidence."""
    articles = gather_news(run) if LIVE else sample_news(run)
    return processed_to_evidence(run, run_chain(articles, nlp))


if LIVE:
    TOOLS = {"earnings": gather_earnings, "news": chained_news, "market":
gather_market}
else:
    TOOLS = {
        "earnings": sample_gatherer("earnings"),
        "news": chained_news,
        "market": sample_gatherer("market"),
    }
```

2. Planner and dynamic tool use

Rubric items: the agent plans its research steps, uses tools dynamically, and routes work to specialists.

2.1 The planner makes a plan

We have two planners. The rule-based one always gives the same three steps in a fixed order. The OpenAI planner asks a model to choose one to three sources and explain why. The model's answer has to fit a fixed format (a list of steps, each with a source and a reason), so a bad answer is rejected instead of breaking the run. If there is no key or the call fails, it falls back to the rule-based plan and records why.

In [4]:
```
from investment_committee.agents.planner import plan_research
from investment_committee.models import Run

SYMBOL = "AAPL"


def plan_table(plan):
    rows = []
    for number, task in enumerate(plan.tasks, start=1):
        rows.append({"step": number, "source": task.source, "reason":
task.reason})
    return table(rows)


demo_run = Run(symbol=SYMBOL)
rules = plan_research(SYMBOL, demo_run.run_id, provider="rules")
print("Rule-based plan:")
```

```
display(plan_table(rules.plan))

model_plan = plan_research(SYMBOL, demo_run.run_id, provider="openai",
settings=settings)
print("Planner asked for:", model_plan.provider_requested, "| used:",
model_plan.provider_used)
if model_plan.fallback_reason:
    print("Fell back because:", model_plan.fallback_reason)
if model_plan.model_call:
    call = model_plan.model_call
    print("Model:", call.model, "| tokens in/out:", call.prompt_tokens,
"/", call.completion_tokens)
plan_table(model_plan.plan)
```

```
Rule-based plan:
```

	step	source	reason
0	1	earnings	Collect the latest quarterly performance.
1	2	news	Collect recent events that may affect the company.
2	3	market	Collect recent price history and market indicators.

```
Planner asked for: openai | used: openai
Model: gpt-6-luna | tokens in/out: 207 / 124
```

Out[4]:

	step	source	reason
0	1	earnings	Review Apple's latest results, guidance, and segment performance to assess fundamentals.
1	2	market	Check AAPL's recent price action, valuation, and trading trends.
2	3	news	Identify recent company developments or industry events that may affect the outlook.

The plan also reacts to memory. If the last run for a stock left a note saying a source was missing, that source moves to the front. Here we hand the planner a pretend note. Section 6 does this for real, with a note written by an actual run.

In [5]:

```
from investment_committee.models import MemoryNote

pretend_note = MemoryNote(
    run_id="earlier-run",
    symbol=SYMBOL,
    confidence=0.4,
    open_gaps=["no news evidence in the pool"],
    lesson="Check the news first next time.",
)
with_note = plan_research(SYMBOL, demo_run.run_id, provider="rules",
previous_note=pretend_note)
plan_table(with_note.plan)
```

Out[5]:

	step	source	reason
0	1	news	Revisit the news gap reported by the previous run.
1	2	earnings	Collect the latest quarterly performance.
2	3	market	Collect recent price history and market indicators.

2.2 Routing: each step goes to its own tool

The router has a simple map from source to tool. To show that it follows the plan, we give it a plan with only a market step and record which tools get called. Only the market tool should run.

In [6]:
```
from investment_committee.agents.planner import ResearchPlan, ResearchTask
from investment_committee.workflows.routing import default_gatherers,
route_plan

for source, tool in default_gatherers().items():
    print(f"{source:9} -> {tool.__module__}.{tool.__name__}")

called = []


def record_calls(source, tool):
    def gather(run):
        called.append(source)
        return tool(run)

    return gather


market_only = ResearchPlan(
    symbol=SYMBOL, tasks=[ResearchTask(source="market", reason="Only check
the price.")]
)
watched = {source: record_calls(source, tool) for source, tool in
TOOLS.items()}
routed = route_plan(Run(symbol=SYMBOL), market_only, gatherers=watched)
print()
print("Tools called:", called)
```

```
earnings  -> investment_committee.tools.earnings.gather_earnings
news      -> investment_committee.tools.news.gather_news
market    -> investment_committee.tools.market.gather_market
Tools called: ['market']
```

2.3 The tools return real data

Now we route the full plan, so all three tools run. Earnings and market data come from yfinance. News comes from yfinance, then NewsAPI, then yfinance search headlines, whichever returns articles first. (Yahoo's ticker news feed stopped working in early

October 2026, which is why the news tool has fallbacks.) News is passed through the news chain from section 3 before it becomes evidence.

In [7]:
```
tool_run = Run(symbol=SYMBOL)
full_plan = plan_research(SYMBOL, tool_run.run_id, provider="rules").plan
gathered = route_plan(tool_run, full_plan, gatherers=TOOLS)


def evidence_table(items):
    rows = []
    for item in items:
        rows.append(
            {
                "source": item.source,
                "title": item.title,
                "content": item.content,
                "as of": item.as_of,
            }
        )
    return table(rows)


for source, items in gathered.items():
    print(f"{source}: {len(items)} evidence records")
evidence_table(gathered["earnings"] + gathered["market"])
```

```
earnings: 3 evidence records
news: 3 evidence records
market: 3 evidence records
```

Out[7]:

	source	title	content	as of
0	earnings	Quarterly revenue	Quarterly revenue was $109.42 billion for the quarter ended 2026-06-30.	2026-06-30
1	earnings	Quarterly net income	Quarterly net income was $29.79 billion for the quarter ended 2026-06-30.	2026-06-30
2	earnings	Diluted EPS	Diluted EPS was $2.02 per share for the quarter ended 2026-06-30.	2026-06-30
3	market	Latest close	AAPL closed at $340.17.	2026-10-12
4	market	RSI(14)	The 14-day RSI was 59.5.	2026-10-12
5	market	MACD(12,26,9)	MACD was 3.57; the signal line was 3.81.	2026-10-12

In [8]:
```
evidence_table(gathered["news"]).head(6)
```

Out[8]:

	source	title	content	as of
0	news	Apple Plans Major iPhone Duo Retail Showcase, But Supply Will Be Limited	Apple is preparing new displays and large numbers of demo units for the iPhone Duo's October 23 launch, but supply will be limited, according to Bloomberg's Mark Gurman. Ahead of the October 23 la...	2026-10-11
1	news	Apple testing two new features for upgraded Apple TV 4K, per report	Apple will hold its "Welcome home" product launch on Tuesday, with the headlining announcement expected to be the new "HomeView" smart display. A new Apple TV 4K is also on the docket for this wee...	2026-10-11
2	news	New Apple TV 4K and HomePod Mini to Ship on October 30	The new upcoming Apple TV 4K and HomePod mini models will launch on October 30, according to Bloomberg's Mark Gurman. Writing in his Power On newsletter, Gurman said the devices will ship to custo...	2026-10-11

2.4 The whole thing in one call

`run_research` does all of the above: read the last memory note, plan, route, run the tools, and save the run and its evidence to SQLite. (We pass in our `TOOLS`, whose news tool runs the news chain with the `nlp` pipeline loaded in setup.) If one source has no data (for example an ETF has no earnings), the run still finishes and says which source was missing. This saved run is the evidence pool the next sections argue from.

In [9]:
```
from investment_committee.storage import load_evidence, open_db
from investment_committee.workflows.runner import run_research

snapshot = run_research(
    SYMBOL,
    gatherers=TOOLS,
    database_path=DB_PATH,
    planner_provider="openai",
)
MAIN_RUN = snapshot.run.run_id

print("Run id:        ", MAIN_RUN)
print("Planner used:  ", snapshot.planner_provider_used,
snapshot.planner_model or "")
print("Plan:          ", [task.source for task in snapshot.plan.tasks])
print("Saved evidence:", snapshot.saved_evidence_count)
for source, message in snapshot.gather_errors.items():
    print(f"Warning ({source}):", plain(message))

with closing(open_db(DB_PATH)) as conn:
    POOL = load_evidence(conn, MAIN_RUN)
table(
    [
        {"source": source, "records": sum(1 for e in POOL if e.source ==
source)}
        for source in ("earnings", "news", "market")
```

```
    ]
)
```

```
Run id:          76680f7e7659
Planner used:    openai gpt-6-luna
Plan:            ['earnings', 'market', 'news']
Saved evidence: 9
```

Out[9]:

	source	records
0	earnings	3
1	news	3
2	market	3

3. News chain: ingest, preprocess, classify, gate, extract, summarize

Rubric item: prompt chaining. Each step's output is the next step's input.

We show the chain on the saved sample articles first, because they include tricky cases on purpose: an exact duplicate, an empty article, HTML tags, and curly quotes.

Step 1: ingest

In [10]:

```
from investment_committee.workflows.chaining import (
    classification_gate,
    classify,
    extract,
    ingest,
    preprocess,
    summarize,
)

articles = ingest(ROOT / "tests/fixtures/sample_articles.json")
table(
    [
        {"id": a.article_id, "symbol": a.symbol, "title": a.title, "body
length": len(a.body)}
        for a in articles
    ]
)
```

Out[10]:

	id	symbol	title	body length
0	a-001	AAPL	Apple beats quarterly earnings expectations	221
1	a-002	AAPL	Apple beats quarterly earnings expectations	227
2	a-003	AAPL	Apple unveils new on-device translation feature	182
3	t-001	TSLA	Regulator opens investigation into Tesla driver-assist claims	197
4	t-002	TSLA	Analyst cuts Tesla price target on margin pressure	144
5	t-003	TSLA	Tesla energy division reports record quarter	199
6	n-001	NFLX	Netflix cafe documentary draws festival praise	165
7	n-002	NFLX	Netflix placeholder item with empty body	10

Step 2: preprocess

Remove HTML, turn curly quotes into plain ones, collapse spaces, and drop empty articles and exact duplicates.

In [11]:
```
kept = preprocess(articles)
kept_ids = [article.article_id for article, _ in kept]
print("In:", len(articles), "| out:", len(kept))
print("Dropped:", [a.article_id for a in articles if a.article_id not in
kept_ids])
print()
# The raw text has a curly apostrophe; backslashreplace prints it as \u2019
so we can see it.
raw = articles[0].body.encode("ascii", "backslashreplace").decode()
print("Before:", raw[60:190])
print("After: ", kept[0][1][60:175])
```

```
In: 8 | out: 6
Dropped: ['a-002', 'n-002']

Before: analyst expectations.<br /><br />Chief executive Tim Cook said the c
ompany\u2019s services business carried the quarter. Guidance
After:  analyst expectations. Chief executive Tim Cook said the company's se
rvices business carried the quarter. Guidance f
```

`a-002` is the same story as `a-001` with extra spaces, and `n-002` has an empty body, so both are dropped. After cleaning, the `<br />` tags are gone and the curly apostrophe (`\u2019`) is a plain `'`.

Steps 3 and 4: classify, then the gate

`classify` gives each article a category from keywords. The **gate** comes right after it: only categories that can move a stock price (earnings, product, legal, market) are allowed into the evidence pool. Articles classified as `general` stop here.

In [12]:
```
rows = []
for article, text in kept:
    category = classify(text)
    accepted, reason = classification_gate(category)
    rows.append(
        {
            "id": article.article_id,
            "title": article.title,
            "category": category,
            "accepted": accepted,
            "gate reason": reason,
        }
    )
table(rows)
```

Out[12]:

	id	title	category	accepted	gate reason
0	a-001	Apple beats quarterly earnings expectations	earnings	True	accepted classification: earnings
1	a-003	Apple unveils new on-device translation feature	product	True	accepted classification: product
2	t-001	Regulator opens investigation into Tesla driver-assist claims	legal	True	accepted classification: legal
3	t-002	Analyst cuts Tesla price target on margin pressure	market	True	accepted classification: market
4	t-003	Tesla energy division reports record quarter	earnings	True	accepted classification: earnings
5	n-001	Netflix cafe documentary draws festival praise	general	False	rejected classification: general

Steps 5 and 6: extract and summarize

`extract` finds named entities (companies, people, money, dates) with spaCy. `summarize` keeps the first two sentences.

In [13]:
```
first_text = kept[0][1]
entities = extract(first_text, nlp)
display(table([{"entity": e.text, "label": e.label} for e in entities]))
print("Summary:", summarize(first_text))
```

	entity	label
0	Apple	ORG
1	quarterly	DATE
2	$91.2 billion	MONEY
3	Tim Cook	PERSON
4	the quarter	DATE
5	the December quarter	DATE

```
Summary: Apple reported quarterly revenue of $91.2 billion, ahead of analyst
expectations. Chief executive Tim Cook said the company's services business
carried the quarter.
```

The whole chain, and the handoff to evidence

`run_chain` runs all the steps. `processed_to_evidence` turns only the **accepted** articles into shared `Evidence` records. Rejected articles stay in the chain output so we can count them, but they never reach the analysts. Evidence belongs to one stock, so here we only hand off the Apple articles.

In [14]:
```
apple_articles = [a for a in articles if a.symbol == "AAPL"]
processed = run_chain(apple_articles, nlp)
sample_run = Run(symbol="AAPL")
news_evidence = processed_to_evidence(sample_run, processed)
print(
    "Processed:",
    len(processed),
    "| accepted:",
    sum(p.accepted for p in processed),
    "| became evidence:",
    len(news_evidence),
)
evidence_table(news_evidence)
```

```
Processed: 2 | accepted: 2 | became evidence: 2
```

Out[14]:

	source	title	content	as of
0	news	Apple beats quarterly earnings expectations	Apple reported quarterly revenue of $91.2 billion, ahead of analyst expectations. Chief executive Tim Cook said the company's services business carried the quarter.	2026-09-08
1	news	Apple unveils new on-device translation feature	At a press event in Cupertino, Apple announced a product update that adds on-device translation. The launch is planned for October, and the feature runs without a network connection.	2026-09-09

The same chain on live news. These are the live articles for our stock. Real news has more noise than the samples, so the gate matters more here.

In [15]:
```
live_articles = gather_news(Run(symbol=SYMBOL)) if LIVE else
sample_news(Run(symbol=SYMBOL))
live_processed = run_chain(live_articles, nlp)
print(
    "Live articles:",
    len(live_articles),
    "| accepted by the gate:",
    sum(p.accepted for p in live_processed),
)
table(
    [
        {"title": p.article.title, "category": p.category, "accepted":
p.accepted}
        for p in live_processed
    ]
)
```

```
Live articles: 9 | accepted by the gate: 3
```

Out[15]:

	title	category	accepted
0	Apple Plans Major iPhone Duo Retail Showcase, But Supply Will Be Limited	product	True
1	Salted Caramel Apple Pie	general	False
2	Beloved wife is crushed to death by falling tree while lying in bed during Hurricane Isaias as trapped husband used Apple Watch to call for help	general	False
3	Apple TV 4K Getting A19 Chip and Easier-to-Find Siri Remote	general	False
4	Apple HomeView reportedly comes in two versions, neither with an App Store	general	False
5	Apple planning huge iPhone Duo demo experiences at select retail stores: report	general	False
6	You can still score Prime Day all-time lows on the latest Apple Watch models, deals from $199	general	False
7	Apple testing two new features for upgraded Apple TV 4K, per report	product	True
8	New Apple TV 4K and HomePod Mini to Ship on October 30	product	True

4. Bull and Bear over the evidence pool

Rubric item: agents that use the evidence to argue a position.

Both analysts use one shared drafting engine with opposite instructions. Each one reads the evidence pool from section 2.4 and must return 1 to 5 claims, and **every claim has to cite evidence ids from the pool**. The prompt also tells the model that the evidence text is data, not instructions (news text could contain anything).

If a claim cites an id that is not in the pool, we keep it on purpose. Throwing it away would hide the mistake. The evaluator in section 5 rejects it with a reason instead.

In [16]:
```
from investment_committee.agents.bear import draft_bear_claims_model
from investment_committee.agents.bull import draft_bull_claims_model

titles = {item.evidence_id: item.title for item in POOL}


def claims_table(claims):
    rows = []
    for claim in claims:
        cited = [titles.get(eid, "NOT IN POOL: " + eid) for eid in
claim.evidence_ids]
        rows.append({"side": claim.side, "claim": claim.text, "cites": ";
".join(cited)})
    return table(rows)


bull = draft_bull_claims_model(POOL, settings, provider=PROVIDER)
bear = draft_bear_claims_model(POOL, settings, provider=PROVIDER)
for name, result in (("Bull", bull), ("Bear", bear)):
    print(
        f"{name}: {len(result.claims)} claims, provider used:
{result.provider_used}",
        f"| unknown ids cited: {result.dropped_ids or 'none'}",
    )
    if result.fallback_reason:
        print("  fell back because:", plain(result.fallback_reason))
claims_table(bull.claims + bear.claims)
```

```
Bull: 3 claims, provider used: openai | unknown ids cited: none
Bear: 3 claims, provider used: openai | unknown ids cited: none
```

Out[16]:

	side	claim	cites
0	bull	Apple's latest quarter shows substantial earnings power, with $109.42 billion in revenue, $29.79 billion in net income, and diluted EPS of $2.02.	Quarterly revenue; Quarterly net income; Diluted EPS
1	bull	Upcoming product activity offers potential catalysts: Apple TV 4K and HomePod mini models are expected to ship October 30, alongside a reported HomeView smart-display announcement.	Apple testing two new features for upgraded Apple TV 4K, per report; New Apple TV 4K and HomePod Mini to Ship on October 30
2	bull	The 14-day RSI of 59.5 is consistent with positive but not extreme recent price momentum.	RSI(14)
3	bear	At $340.17, AAPL trades at roughly 42 times the quarterly EPS annualized ($2.02 4). That simple run-rate comparison suggests the stock may be vulnerable if earnings growth fails to justify a high ...	Latest close; Diluted EPS
4	bear	The MACD reading of 3.57 is below its 3.81 signal line, a potentially bearish momentum signal that may point to near-term weakness.	MACD(12,26,9)
5	bear	Supply for the iPhone Duo is expected to be limited at launch, which could constrain near-term availability and sales even as Apple prepares a major retail showcase.	Apple Plans Major iPhone Duo Retail Showcase, But Supply Will Be Limited

5. Evaluator-optimizer loop and the verdict

Rubric items: the agent self-reflects on the quality of its output, and the evaluator-optimizer workflow.

The loop works like this:

1. Bull and Bear draft claims (section 4).
2. The **evaluator** checks every claim in two gates. The rules gate rejects any claim that cites evidence not in the pool. The model gate reads each remaining claim next to the text of the evidence it cites and decides if that text really supports it, with a one-sentence reason.
3. Rejected claims go **back to their analyst with the reason**, and the analyst writes a replacement. Supported claims are kept.
4. This repeats until everything is supported or we hit the round limit (2 rounds).
5. A final call reads **only the supported claims** and picks bull, bear, or none. We do not just count claims, because then the side that writes more small claims would win.

We run the committee as a new run that points to the evidence run from section 2.4 (its "parent"), so it argues from exactly that pool.

In [17]:
```
from investment_committee.storage import save_run
from investment_committee.workflows.evaluator_optimizer import
run_committee_from_storage

with closing(open_db(DB_PATH)) as conn:
    committee_run = Run(symbol=SYMBOL, with_review=True,
parent_run_id=MAIN_RUN)
    save_run(conn, committee_run)
    result = run_committee_from_storage(conn, committee_run.run_id,
settings, provider=PROVIDER)

print("Rounds:", result.rounds, "| verdict:", result.verdict)
print("Fallbacks:", result.fallbacks or "none")
```

```
Rounds: 2 | verdict: none
Fallbacks: none
```

Every claim the evaluator saw, round by round. Revision 0 is the first draft. Revision 1 claims are rewrites of claims that were rejected in round 1.

In [18]:
```
rows = []
for claim in result.all_claims:
    rows.append(
        {
            "revision": claim.revision,
            "side": claim.side,
            "supported": claim.supported,
            "evaluator reason": claim.reason,
            "claim": claim.text,
        }
    )
table(rows)
```

Out[18]:

	revision	side	supported	evaluator reason	claim
0	0	bull	True	The evidence states quarterly revenue of $109.42 billion and net income of $29.79 billion for the quarter ended June 30, 2026, supporting the claim of substantial scale and profitability.	Apple reported $109.42 billion in quarterly revenue and $29.79 billion in net income for the quarter ended June 30, 2026, demonstrating substantial business scale and profitability.
1	0	bull	True	The cited evidence states that diluted EPS was $2.02 per share for the quarter, supporting the claim that this figure is an indicator of earnings generation.	Quarterly diluted EPS of $2.02 adds another positive indicator of the company's earnings generation.
2	0	bull	False	The evidence supports the planned iPhone Duo showcase, limited supply, and upcoming Apple TV 4K and HomePod mini models, but it does not establish that these touchpoints could sustain engagement a...	Apple has several upcoming product touchpoints-including the iPhone Duo retail showcase and new Apple TV 4K and HomePod mini models-which could help sustain customer engagement across its device e...
3	0	bear	True	The evidence states that supply will be limited ahead of the October 23 launch, supporting the claim that constrained availability could limit near-term sales despite launch interest.	Limited supply for the iPhone Duo's October 23 launch could constrain Apple's ability to convert launch interest into near-term sales.
4	0	bear	True	The evidence states that MACD was 3.57 and the signal line was 3.81, so MACD is below the signal line despite being positive.	AAPL's MACD of 3.57 is below its 3.81 signal line, a bearish momentum signal despite the positive MACD reading.
5	1	bull	True	The cited reports identify the iPhone Duo's October 23 launch, an event expected to feature the HomeView, and Apple TV 4K and HomePod mini models shipping October 30, while providing no sales or d...	Apple has several identifiable near-term product milestones: an iPhone Duo launch planned for October 23, a product event expected to feature the HomeView smart display, and new Apple TV 4K and Ho...
6	0	bull	True	The cited evidence directly reports $109.42 billion in quarterly revenue and $29.79 billion in net income for the quarter ended June 30, 2026, supporting the	Apple reported $109.42 billion in quarterly revenue and $29.79 billion in net income for the quarter ended June 30, 2026, demonstrating substantial

	revision	side	supported	evaluator reason	claim
				claim of substantial scale and profita...	business scale and profitability.
7	0	bull	True	The evidence states that diluted EPS was $2.02 per share for the quarter, supporting the stated earnings-generation indicator.	Quarterly diluted EPS of $2.02 adds another positive indicator of the company's earnings generation.
8	0	bear	True	The cited evidence says supply will be limited for the October 23 launch, which could constrain Apple's ability to meet launch demand and convert interest into near-term sales.	Limited supply for the iPhone Duo's October 23 launch could constrain Apple's ability to convert launch interest into near-term sales.
9	0	bear	True	The evidence states that MACD was 3.57 and the signal line was 3.81, confirming that MACD was below the signal line; the positive reading does not contradict that comparison.	AAPL's MACD of 3.57 is below its 3.81 signal line, a bearish momentum signal despite the positive MACD reading.

The memo. The verdict comes first, then the diagnostics, then the supported claims with their evidence ids.

In [19]:
```
print(plain(result.memo))
```

```
Verdict: none. Apple's substantial revenue and profits plus upcoming launche
s offer support, but limited iPhone supply and a mildly bearish MACD offset
these signals, while the evidence provides little context on growth, valuati
on, or expected demand.
Diagnostics: 3 supported bull claim(s), 2 supported bear claim(s) after 2 ro
und(s); counts informed nothing.
- [bull] Apple has several identifiable near-term product milestones: an iPh
one Duo launch planned for October 23, a product event expected to feature t
he HomeView smart display, and new Apple TV 4K and HomePod mini models sched
uled to ship October 30. This active launch calendar offers potential cataly
sts, although the reports do not establish expected sales or demand. (eviden
ce: 27e892df1147, f99a941df1bc, 6301ea8e853e)
- [bull] Apple reported $109.42 billion in quarterly revenue and $29.79 bill
ion in net income for the quarter ended June 30, 2026, demonstrating substan
tial business scale and profitability. (evidence: 4e2010bda5e5, 59146655cbd
f)
- [bull] Quarterly diluted EPS of $2.02 adds another positive indicator of t
he company's earnings generation. (evidence: 52925fa77ccb)
- [bear] Limited supply for the iPhone Duo's October 23 launch could constra
in Apple's ability to convert launch interest into near-term sales. (evidenc
e: 27e892df1147)
- [bear] AAPL's MACD of 3.57 is below its 3.81 signal line, a bearish moment
um signal despite the positive MACD reading. (evidence: d379dccb26af)
```

Self-reflection. At the end, the committee scores its own work: its confidence is the share of final claims that were supported, and its open gaps list what it went without (a missing source, or claims that were sent back). This becomes the memory note for the next run (section 6).

We also keep every model call with its stage, tokens, and time, so we can see what the loop cost.

In [20]:
```python
print("Confidence:", round(result.note.confidence, 2))
print("Open gaps: ", result.note.open_gaps or "none")
print("Lesson:    ", plain(result.note.lesson))
print()
calls = pd.DataFrame(
    [
        {"stage": c.stage, "tokens": c.prompt_tokens + c.completion_tokens,
"ms": c.latency_ms}
        for c in result.model_calls
    ]
)
if calls.empty:
    print("No model calls (rule-based run).")
else:
    display(
        calls.groupby("stage").agg(
            calls=("tokens", "size"), tokens=("tokens", "sum"), total_ms=
("ms", "sum")
        )
    )
```

```
Confidence: 1.0
Open gaps:  none
Lesson:     Verdict none after 2 round(s): Apple's substantial revenue and p
rofits plus upcoming launches offer support, but limited iPhone supply and a
mildly bearish MACD offset these signals, while the evidence provides little
context on growth, valuation, or expected demand.
```

stage	calls	tokens	total_ms
bear	1	1097	4221
bull	2	2730	12815
evaluator	10	3794	27974
verdict	1	1152	3090

A controlled example of one full review cycle

A live run does not always reject a claim, so here is a small controlled example where we know a rejection will happen. Instead of OpenAI, we plug in a **scripted client** that gives fixed answers in the same format the model would. This is the same trick our tests use. Everything else (the loop, the two evaluator gates, the rewrite, the verdict) is the real code.

The script: the Bull analyst cites real evidence. The Bear analyst's first claim cites an evidence id that does not exist, so the rules gate rejects it without even asking the model. The Bear analyst then gets the rejection reason back and writes a replacement that cites real evidence, and round 2 accepts it.

This is not a live result. The answers are written by us to show the mechanism.

In [21]:
```python
from types import SimpleNamespace

from investment_committee.agents.analyst import DraftedClaim, DraftedClaims
from investment_committee.agents.evaluator import ClaimVerdict
from investment_committee.config import ENV_VARS, Settings
from investment_committee.workflows.evaluator_optimizer import
CommitteeVerdict, run_committee


class ScriptedClient:
    """Stands in for the OpenAI client and returns our fixed answers in
order."""

    def __init__(self, drafts, verdicts, decisions):
        self.answers = {DraftedClaims: drafts, ClaimVerdict: verdicts,
CommitteeVerdict: decisions}
        self.responses = self
```

```
    def parse(self, *, model, instructions, input, text_format):
        answer = self.answers[text_format].pop(0)
        usage = SimpleNamespace(input_tokens=0, output_tokens=0)
        return SimpleNamespace(output_parsed=answer, model="scripted",
usage=usage)


def one_claim(evidence_id, text):
    return DraftedClaims(claims=[DraftedClaim(text=text, evidence_ids=
[evidence_id])])


script = ScriptedClient(
    drafts=[
        one_claim("ev-001", "Revenue grew 4 percent from a year earlier."),
# Bull, round 1
        one_claim("ev-999", "Profit collapsed this quarter."),  # Bear,
round 1: no such id
        one_claim("ev-002", "EPS beat consensus by only 4 cents."),  # Bear
rewrite, round 2
    ],
    verdicts=[
        ClaimVerdict(supported=True, reason="The revenue row says up 4
percent."),
        ClaimVerdict(supported=True, reason="Still matches the revenue
row."),
        ClaimVerdict(supported=True, reason="EPS was 1.42 against 1.38."),
    ],
    decisions=[CommitteeVerdict(side="bull", reason="Growth is clear; the
EPS beat is small.")],
)

sample_pool = [Evidence(**item) for item in SAMPLE_EVIDENCE["evidence"]]
scripted_settings = Settings(
    values={name: "scripted" if name == "OPENAI_API_KEY" else None for name
in ENV_VARS}
)
controlled = run_committee(
    Run(symbol="AAPL", with_review=True), sample_pool, scripted_settings,
client=script
)

table(
    [
        {
            "revision": c.revision,
            "side": c.side,
            "supported": c.supported,
            "evaluator reason": c.reason,
            "evidence": ", ".join(c.evidence_ids),
            "claim": c.text,
        }
        for c in controlled.all_claims
    ]
)
```

Out[21]:

	revision	side	supported	evaluator reason	evidence	claim
0	0	bull	True	The revenue row says up 4 percent.	ev-001	Revenue grew 4 percent from a year earlier.
1	0	bear	False	cites evidence not in the pool: ev-999	ev-999	Profit collapsed this quarter.
2	0	bull	True	Still matches the revenue row.	ev-001	Revenue grew 4 percent from a year earlier.
3	1	bear	True	EPS was 1.42 against 1.38.	ev-002	EPS beat consensus by only 4 cents.

Round 1 rejected the Bear claim with the reason "cites evidence not in the pool: ev-999". In round 2 the Bear analyst replaced it with a claim that cites `ev-002`, the evaluator accepted it, and the kept Bull claim was checked again. Then the loop stopped because nothing was left unsupported.

In [22]:
```
print("Rounds:", controlled.rounds, "| verdict:", controlled.verdict)
print(controlled.memo)
```

```
Rounds: 2 | verdict: bull
Verdict: bull. Growth is clear; the EPS beat is small.
Diagnostics: 1 supported bull claim(s), 1 supported bear claim(s) after 2 ro
und(s); counts informed nothing.
- [bull] Revenue grew 4 percent from a year earlier. (evidence: ev-001)
- [bear] EPS beat consensus by only 4 cents. (evidence: ev-002)
```

6. Memory across runs

Rubric item: the agent learns across runs.

After every committee run, its memory note (confidence, open gaps, lesson) is saved in SQLite. When the next run for the same stock starts, `run_research` loads the latest note and gives it to the planner.

To make the effect easy to see, we use a different stock and pretend the market tool is down in the first run. The committee should notice that it had no market evidence, and the second run's planner should move market from the last step to the first.

In [23]:
```
from investment_committee.storage import load_latest_note

MEMORY_SYMBOL = "MSFT"
TOOLS_NO_MARKET = {**TOOLS, "market": lambda run: []}

with closing(open_db(DB_PATH)) as conn:
    print("Note before run 1:", load_latest_note(conn, MEMORY_SYMBOL))

first = run_research(
```

```
    MEMORY_SYMBOL, gatherers=TOOLS_NO_MARKET, database_path=DB_PATH,
planner_provider="rules"
)
print("Run 1 plan:", [task.source for task in first.plan.tasks])

with closing(open_db(DB_PATH)) as conn:
    first_committee = run_committee_from_storage(
        conn, first.run.run_id, settings, provider=PROVIDER
    )
    saved_note = load_latest_note(conn, MEMORY_SYMBOL)

print("Run 1 verdict:", first_committee.verdict)
print("Saved note -> confidence:", round(saved_note.confidence, 2))
print("              open gaps:", saved_note.open_gaps)
```

```
Note before run 1: None
Run 1 plan: ['earnings', 'news', 'market']
Run 1 verdict: bull
Saved note -> confidence: 1.0
              open gaps: ['no market evidence in the pool']
```

Now the second run. Nothing is passed in by hand: `run_research` reads the note from the database by itself.

In [24]:

```
second = run_research(
    MEMORY_SYMBOL, gatherers=TOOLS, database_path=DB_PATH,
planner_provider="rules"
)
print("Lesson the planner read:", plain(second.plan.memory_lesson))
display(
    table(
        [
            {"step": n, "run 1": a.source, "run 2": b.source, "run 2
reason": b.reason}
            for n, (a, b) in enumerate(
                zip(first.plan.tasks, second.plan.tasks, strict=True),
start=1
            )
        ]
    )
)
```

```
Lesson the planner read: Verdict bull after 2 round(s): Reported revenue of
$90.01 billion, net income of $35.77 billion, and diluted EPS of $4.81 provi
de substantial evidence of financial strength, while the bear concerns are l
imited to isolated product-usability and promotional examples.
```

	step	run 1	run 2	run 2 reason
0	1	earnings	market	Revisit the market gap reported by the previous run.
1	2	news	earnings	Collect the latest quarterly performance.
2	3	market	news	Collect recent events that may affect the company.

Market moved from step 3 to step 1, and its reason now says it is revisiting the gap from the last run. Nobody changed the code or the settings between the two runs; the only difference is the note the first run left behind.

The OpenAI planner gets the same note inside its prompt, so it can use it too:

In [25]:
```
model_with_memory = plan_research(
    MEMORY_SYMBOL, second.run.run_id, provider="openai",
previous_note=saved_note, settings=settings
)
print("Planner used:", model_with_memory.provider_used)
plan_table(model_with_memory.plan)
```

```
Planner used: openai
```

Out[25]:

	step	source	reason
0	1	market	Address the open gap by checking MSFT's current price performance and market valuation.
1	2	news	Identify recent developments that could affect the outlook or challenge the prior bullish assessment.

7. Evaluation and iteration: with vs without review

Question: does the evaluator loop make the committee's final claims better supported?

How we test it fairly. For each stock we gather evidence **once**. Then we run the committee twice on that same saved evidence: once with the review loop and once without it (draft, judge once, decide). Both runs point to the same parent run in SQLite, so the evidence is identical and the review loop is the only setting we change. Each run still writes its own first draft, so the drafts can differ too (see the limits below).

7.1 A controlled comparison (scripted, offline)

First the same experiment with the scripted client from section 5, so the difference between the arms is easy to see and the result is the same every time. Both arms get the same scripted first drafts: a good Bull claim and a Bear claim that cites a missing id. With review, the Bear claim is sent back and rewritten. Without review, it stays rejected. **This shows how the experiment works, not whether the loop helps:** we wrote these answers ourselves.

In [26]:
```
from investment_committee.evaluation.harness import (
    plot_support_comparison,
    run_review_comparison,
    run_symbol_sweep,
)
from investment_committee.evaluation.metrics import comparison_rows,
score_run
```

```
from investment_committee.storage import save_run_with_evidence

with_review_answers = dict(
    drafts=[
        one_claim("ev-001", "Revenue grew 4 percent from a year earlier."),
        one_claim("ev-999", "Profit collapsed this quarter."),
        one_claim("ev-002", "EPS beat consensus by only 4 cents."),
    ],
    verdicts=[ClaimVerdict(supported=True, reason="Matches the cited
row.")] * 3,
    decisions=[CommitteeVerdict(side="none", reason="Each side has one
grounded point.")],
)
without_review_answers = dict(
    drafts=[
        one_claim("ev-001", "Revenue grew 4 percent from a year earlier."),
        one_claim("ev-999", "Profit collapsed this quarter."),
    ],
    verdicts=[ClaimVerdict(supported=True, reason="Matches the cited
row.")],
    decisions=[CommitteeVerdict(side="bull", reason="Only the Bull claim
survived.")],
)
# run_review_comparison runs the with-review arm first, then the other one.
script = ScriptedClient(
    drafts=with_review_answers["drafts"] +
without_review_answers["drafts"],
    verdicts=with_review_answers["verdicts"] +
without_review_answers["verdicts"],
    decisions=with_review_answers["decisions"] +
without_review_answers["decisions"],
)

controlled_conn = open_db(":memory:")
controlled_parent = Run(**SAMPLE_EVIDENCE["run"])
save_run_with_evidence(controlled_conn, controlled_parent, sample_pool)
controlled_ab = run_review_comparison(
    controlled_conn, controlled_parent.run_id, scripted_settings,
client=script
)
controlled_ids = [controlled_ab.with_review.run.run_id,
controlled_ab.without_review.run.run_id]
table(comparison_rows(controlled_conn, controlled_ids))
```

Out[26]:

	symbol	review loop	verdict	rounds	supported claims	rejected drafts	model calls	tokens	run i
0	AAPL	on	none	2	2	1	7	0	90fd3b4ad5b
1	AAPL	off	bull	1	1	1	4	0	ad51f85d5e0

With review, the Bear claim was rewritten and both sides reached the chair, so the verdict was "none". Without review, only the Bull claim survived, so the chair said "bull". The loop changed which arguments the decision was based on.

7.2 The live comparison

What we measure for each run:

- **Supported claims:** how many grounded claims the evaluator saved for the chair.
- **Rejected drafts:** how many drafts were sent back or left out of the verdict.
- Rounds, model calls, and tokens: what the loop costs.
- Verdict: did the loop change the decision?

In [27]:
```
EVAL_SYMBOLS = ["AAPL", "NVDA", "JPM"]

# Michael's sweep helper gathers each stock once, runs both review arms
from
# that saved evidence, and reads the scorecard back from SQLite.
sweep_rows = run_symbol_sweep(
    EVAL_SYMBOLS,
    DB_PATH,
    settings,
    gatherers=None if LIVE else TOOLS,
    planner_provider="rules",
    provider=PROVIDER,
)
sweep_table = table(sweep_rows)
if PROVIDER == "template":
    print("Rule-based run: the template analysts cite one real item per
claim, so every claim")
    print("passes and both arms look the same. The comparison needs the
model run.")
sweep_table.drop(columns=["run id"])
```

Out[27]:

	symbol	review loop	verdict	rounds	supported claims	rejected drafts	model calls	tokens
0	AAPL	on	bull	2	4	2	14	8695
1	AAPL	off	bear	1	3	1	7	4694
2	NVDA	on	bull	2	3	2	11	8790
3	NVDA	off	none	1	3	2	8	5871
4	JPM	on	none	2	2	1	8	5207
5	JPM	off	none	1	3	0	6	3759

In [28]:
```
columns = ["supported claims", "rejected drafts", "model calls", "rounds"]
summary = sweep_table.groupby("review loop")[columns].mean()
summary.round(2)
```

Out[28]:

review loop	supported claims	rejected drafts	model calls	rounds
off	3.0	1.00	7.0	1.0
on	3.0	1.67	11.0	2.0

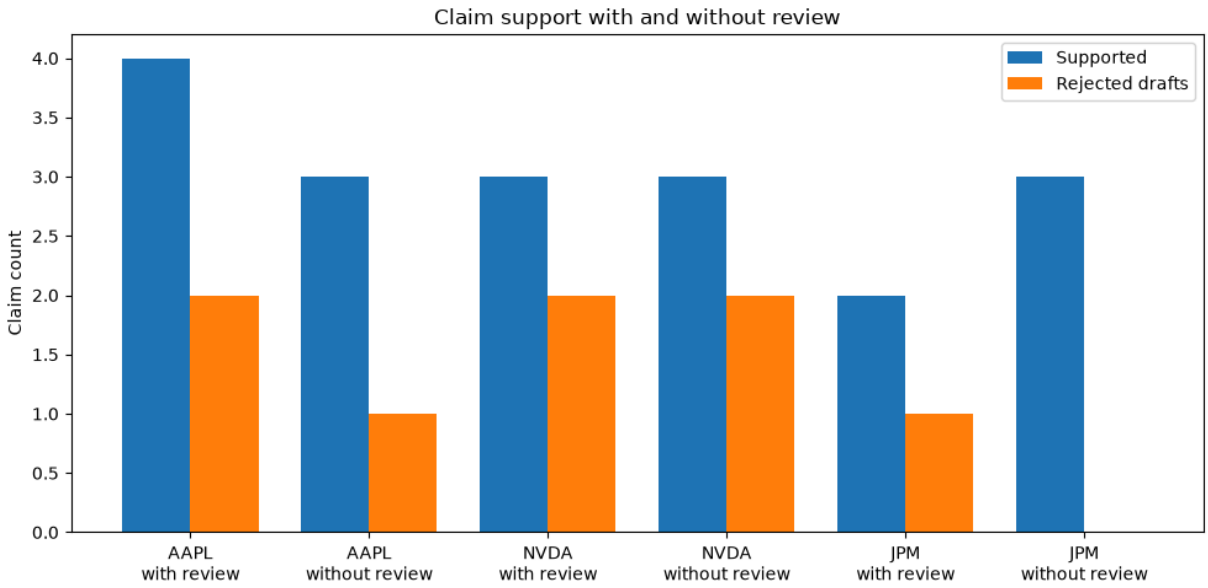
The stored scorecard. `run_symbol_sweep` already returns `comparison_rows` from SQLite. Here "supported claims" and "rejected drafts" count every draft the evaluator judged, including drafts the loop later replaced, so they show how much work the loop did.

In [29]:
```
run_ids = [row["run id"] for row in sweep_rows]

with closing(open_db(DB_PATH)) as conn:
    scores = [score_run(conn, run_id) for run_id in run_ids]
display(sweep_table)

figure, axis = plot_support_comparison(scores)
```

	symbol	review loop	verdict	rounds	supported claims	rejected drafts	model calls	tokens	run id
0	AAPL	on	bull	2	4	2	14	8695	e48354b3074b
1	AAPL	off	bear	1	3	1	7	4694	0ecdba539c40
2	NVDA	on	bull	2	3	2	11	8790	caa8ac598326
3	NVDA	off	none	1	3	2	8	5871	251df7b18489
4	JPM	on	none	2	2	1	8	5207	fa5c8d76acde
5	JPM	off	none	1	3	0	6	3759	78d685277c83



How to read this. Without review, the first drafts are judged once and go straight to the verdict. With review, rejected claims are sent back with the evaluator's reason and rewritten, and the rewrites are judged again. A rewrite can be rejected too, and after the second round the loop stops, so the final slate with review can still contain unsupported claims. The chair only reads the supported ones either way. The chart shows the extra work the loop does: rejected drafts are claims the evaluator sent back.

Limits of this test. Each arm writes its own first draft, so some of the difference between arms comes from the drafts and not from the loop. Three stocks is a small sample. And the same model family drafts and judges the claims, and model judges tend to favor their own writing (Panickssery et al., 2024).

8. Conclusions

These points are based on the saved run above. Live data and model answers change between runs, so the exact numbers will not repeat.

What worked

- The planner, tools, and routing worked on live data for every stock we tried. The fallbacks kept everything running when Yahoo's news feed broke and when there was no API key.
- Every claim in every memo cites evidence ids, and the evaluator gives a reason for every claim it judges, so each verdict can be traced back to the data.
- The news gate accepted three of nine live articles and blocked six weak or unrelated Apple mentions before they reached the analysts.
- Memory worked. A run with no market data left a note saying so, and the next run moved market from step 3 to step 1 by itself. The OpenAI planner read the same note and also put market first.
- The controlled example showed the loop clearly: an unsupported Bear draft was rejected, rewritten with real evidence, and accepted in round 2.
- In the live sweep, both arms averaged 3.0 supported claims. Review changed the AAPL verdict from bear to bull and the NVDA verdict from none to bull; JPM stayed at none.

What did not work as well

- The loop is not free: it used 11 model calls and 2 rounds per stock on average, compared with 7 calls and 1 round without review.
- Review did not increase the average supported-claim count in this run. It also produced more rejected drafts because rewritten claims were judged again.
- The results move between runs, and three stocks are too small a sample for a strong conclusion.

- Each arm writes its own first draft, so verdict differences cannot be credited only to the review loop.
- The evaluator is conservative. It rejected a claim about future customer engagement because the cited product announcements did not provide evidence about that outcome.
- The news gate is keyword based. It removes obvious noise, but it cannot judge relevance as carefully as a model or a person. It checks what kind of story an article is, not whether the story is about the company we are researching.

What this does not show

- These results are about whether the committee's claims are supported by the evidence it cites. They do **not** show that the verdicts predict stock prices or returns. We never tested that, and a bull or bear verdict here is not a trading signal.

With more time

- Share one first draft between both arms, so the loop is the only difference, and run more stocks several times each.
- Use a judge from a different model family than the analysts (Panickssery et al., 2024), or check claims with an NLI model.
- Replace the keyword classifier with a model, and add SEC filings for the earnings analyst.

9. AI tool disclosures

Each of us writes our own paragraph in the first person, following `docs/AI_DISCLOSURE.md`: which tools we used, what they did, what we changed or checked, and that the work reflects our own understanding.

Guna Pasupathy:

- Used Grammarly to update the sections of this notebook.
- Used Codex for reviewing the python code I wrote and asked for critical feedback. I incorporated a lot of its suggestions which I missed at first, like the fallback news reader feature.
- Used ChatGPT to format the text diagram in section 1.

Michael Valderrama. I used Claude, a large language model developed by Anthropic (Anthropic, 2026), as a tool to aid my understanding of the course material, mostly through Claude Code. It helped me analyze different approaches and drafted the code and tests for my parts of the project: the Bull and Bear analysts, the evaluator, the evaluator-optimizer loop, the with and without review harness, and the multi-symbol sweep. I have reviewed and verified all AI-generated content against the cited sources,

and the methodology, claims, and conclusions presented reflect my own understanding of the material.

Mina Habib. TODO(mina)

References

Anthropic. (2024). *Building effective agents*. https://www.anthropic.com/research/building-effective-agents

Anthropic. (2026). *Claude* (Fable 5.1) [Large language model]. https://claude.ai

Aroussi, R. (2026). *yfinance* (Version 1.7.0) [Computer software]. https://github.com/ranaroussi/yfinance

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). *Improving factuality and reasoning in language models through multiagent debate*. International Conference on Machine Learning. https://arxiv.org/abs/2305.14325

Grammarly. (2026). *Grammarly* [Computer software]. https://www.grammarly.com

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength natural language processing in Python*. Zenodo. https://doi.org/10.5281/zenodo.1212303

NewsAPI. (n.d.). *News API*. Retrieved October 10, 2026, from https://newsapi.org

OpenAI. (2026a). *ChatGPT* [Large language model]. https://chatgpt.com

OpenAI. (2026b). *Codex* [Computer software]. https://openai.com/codex

OpenAI. (2026c). *GPT-6 Luna* [Large language model]. https://platform.openai.com/docs/models

Panickssery, A., Bowman, S. R., & Feng, S. (2024). *LLM evaluators recognize and favor their own generations*. Advances in Neural Information Processing Systems. https://arxiv.org/abs/2404.13076

Xiao, Y., Sun, E., Luo, D., & Wang, W. (2024). *TradingAgents: Multi-agents LLM financial trading framework*. https://arxiv.org/abs/2412.20138